

Decision Tree Review and Example

1 What is a Decision Tree?

A decision tree is a mathematical model that learns how to make decisions by finding a sequence of conditions that separates different outcomes. It works by repeatedly dividing a dataset into smaller groups based on the values of different features, creating a structured process for reaching a decision. At each step, the model selects the condition that provides the most useful separation between the possible outcomes. By applying a series of these conditions, the model gradually narrows down the possibilities until it can make a final prediction or classification.

1.1 motivation

The basic idea behind a decision tree can be understood through a simple example.

Assume you are an HR person hiring for a position that requires experience in both Python and C programming. There is no strict cutoff for the number of years of experience required.

On your desk, you have 100 resumes, and your goal is to identify the best candidates to interview. You start by creating a simple rule. For example, you choose a cutoff of 5 years of experience in C programming. You review all resumes and divide them into two groups: those with more than 5 years of C experience and those with less than or equal to 5 years.

You then take the group with more than 5 years of C experience and apply another condition. This time, you look at Python experience. Candidates with more than 5 years of Python experience are placed into one group, while the remaining candidates are placed into another group.

After these two decisions, you have created a smaller group of candidates that better matches the requirements. You can now focus your attention on this group when selecting candidates for interviews.

Note that the final set might still be too large. Then we might have to add another condition, like education for example. That split that set one more time giving us a smaller set. Or we can increase the years of experience, to get fewer candidates. The main idea is that each decision reduces the size of the group and moves us closer to the set of candidates we are looking for.

A decision tree learns this process from data instead of requiring a person to choose the rules manually. To make the process mathematical, we use historical data. For example, we can look at previous employees and identify which candidates remained with the company for 5 or more years or, which candidates were promoted within two years (maybe both). We label those as "hire" and the rest as "pass". This has to be yes/no kind of decision always.

So to set up the model, we are going to start from a very large table with 3 columns, years of experience in C, years of experience in Python, and a last column of "hire/pass". The model then examines the table and tell us which features are more important and what threshold we need to apply to end up with a small pool that ideally has all "hire" but in reality will have way more hire than pass.

2 From Decision Rules to a Mathematical Model

In the previous example, we manually created a sequence of decisions to separate candidates into groups. Each decision used a criterion about the candidate and created a new subset of the data. In machine learning terminology, the criterion used to make a decision is called a **feature**. In our example, the number of years of experience in C and Python are features:

$$x = (x_1, x_2)$$

where:

$$x_1 = \text{years of experience in C} \quad \text{and} \quad x_2 = \text{years of experience in Python}$$

The final outcome we are trying to predict is called the **target** or **label**. In this example, the label could represent whether a candidate became a successful employee:

$$y = \begin{cases} 1, & \text{candidate is considered successful} \\ 0, & \text{candidate is not considered successful} \end{cases}$$

The purpose of the decision tree is to learn a relationship between the input information x and the outcome y .

$$\hat{y} = T(x)$$

where:

$$T(x) = \text{the decision rules learned by the tree}$$

and:

$$\hat{y} = \text{the predicted outcome}$$

The individual decisions we made in the hiring example are called **splits**. A split is a rule that divides the data into smaller groups based on the value of one feature.

For example:

$$x_1 > 5$$

means:

$$\text{years of C experience} > 5$$

This split divides the candidates into two groups:

$$\begin{cases} x_1 > 5 & \rightarrow \text{Group A} \\ x_1 \leq 5 & \rightarrow \text{Group B} \end{cases}$$

Each split creates branches in the tree. As more decisions are made, the tree grows deeper (or taller) and creates more specific groups.

The starting point of the tree is called the **root node**. The intermediate decision points are called **internal nodes**, and the final groups where a prediction is made are called **leaf nodes**.

The complete tree can therefore be viewed as a collection of decision rules:

$$T(x) = \begin{cases} \text{prediction 1,} & \text{if conditions are satisfied} \\ \text{prediction 2,} & \text{otherwise} \end{cases}$$

The central problem in building a decision tree is determining which feature to split on, what threshold to use, and when to stop splitting. The mathematics of decision trees provides methods for making these choices automatically.

2.1 How Does a Decision Tree Choose the Best Split?

A good split should create groups where the outcomes are as similar as possible. In other words, after making a split, we want each group to contain examples that mostly belong to the same category. Consider what happens if a split creates two groups:

Group A	Group B
Mostly potential candidates	Mostly not candidates

This is a useful split because knowing which group an observation (resume) belongs to gives us a strong indication of the final outcome (candidate or not). On the other hand, a poor split would create groups where successful and unsuccessful outcomes are mixed together. The split did not provide much useful information.

The concept of measuring how mixed or uncertain a group is called **impurity**. A decision tree uses an impurity measure to evaluate possible splits. The goal is to find the split that produces the largest reduction in impurity. For a dataset before a split, we can measure its impurity. After applying a possible split, we measure the impurity of the resulting groups. The quality of the split is determined by how much the impurity decreases:

$$\text{Improvement} = \text{Impurity before split} - \text{Impurity after split}$$

A larger reduction means the split created cleaner groups and is considered a better decision rule. There are several mathematical measures used to quantify impurity. The two most common measures used in decision trees are: **Gini Impurity** and **Entropy**. Both measures answer the same fundamental question:

How much uncertainty remains after this decision?

The decision tree evaluates many possible splits and selects the one that provides the largest improvement. This process is repeated at each level of the tree until a stopping condition is reached.

2.2 Gini Impurity

The Gini impurity of a node measures the probability of incorrectly classifying a randomly selected sample if the class label is assigned according to the class distribution within that node.

For a node containing K classes, the Gini impurity is defined as:

$$G = 1 - \sum_{i=1}^K p_i^2$$

where:

- G is the Gini impurity of the node.
- K is the number of classes.
- p_i is the proportion of samples belonging to class i .

The term p_i^2 represents the probability that two independent selections belong to the same class i :

$$p_i^2 = p_i \times p_i$$

where the first p_i represents the probability of selecting class i in the first selection, and the second p_i represents the probability of selecting class i in the second selection.

Therefore:

$$\sum_{i=1}^K p_i^2$$

represents the total probability that two independent selections belong to the same class. The Gini impurity is the complement of this probability:

$$G = 1 - \sum_{i=1}^K p_i^2$$

which represents the probability that the two selections belong to different classes. The class probability is:

$$p_i = \frac{N_i}{N}$$

where:

- N_i is the number of samples belonging to class i .
- N is the total number of samples in the node.

2.2.1 Binary Classification Case

For a binary classification problem with class probabilities p and $1 - p$:

$$G = 1 - p^2 - (1 - p)^2$$

which simplifies to:

$$G = 2p(1 - p)$$

2.2.2 Gini Impurity Before and After a Split

When a decision tree considers a split, it compares the impurity before the split and after the split.

Before the split:

$$G_{\text{before}} = 1 - \sum_{i=1}^K p_i^2$$

After the split, the impurity is the weighted average of the child node impurities:

$$G_{\text{after}} = \frac{N_L}{N} G_L + \frac{N_R}{N} G_R$$

where:

- N_L, N_R are the number of samples in the left and right child nodes.
- G_L, G_R are the Gini impurities of the left and right child nodes.

The reduction in impurity is:

$$\Delta G = G_{\text{before}} - G_{\text{after}}$$

where ΔG is the Gini gain.

A decision tree selects the split that maximizes ΔG , producing child nodes with lower impurity.

2.3 Entropy and Information Gain

2.3.1 Entropy

Entropy is a measure of uncertainty in a node of a decision tree. It quantifies the amount of disorder in the class distribution.

For a node containing K classes, the entropy is defined as:

$$H = - \sum_{i=1}^K p_i \log_2(p_i)$$

where:

- H is the entropy of the node.
- K is the number of classes.
- p_i is the probability (proportion) of samples belonging to class i .

The term:

$$-p_i \log_2(p_i)$$

represents the contribution of class i to the total uncertainty.

The logarithm appears because entropy is derived from information theory. The information associated with an event of probability p_i is:

$$I(p_i) = -\log_2(p_i)$$

Entropy is the expected information:

$$H = \sum_{i=1}^K p_i I(p_i)$$

which gives:

$$H = - \sum_{i=1}^K p_i \log_2(p_i)$$

2.3.2 Interpretation of Entropy

If a node contains samples from only one class:

$$H = 0$$

The node has no uncertainty.

If all classes are equally represented:

$$p_i = \frac{1}{K}$$

entropy reaches its maximum value:

$$H_{\max} = \log_2(K)$$

The node has maximum uncertainty.

2.3.3 Entropy Before and After a Split

For a decision tree split, the entropy before splitting is the entropy of the parent node:

$$H_{\text{before}} = - \sum_{i=1}^K p_i \log_2(p_i)$$

After splitting, the data is divided into child nodes. The remaining entropy is the weighted average of the child entropies:

$$H_{\text{after}} = \frac{N_L}{N} H_L + \frac{N_R}{N} H_R$$

where:

- N_L and N_R are the numbers of samples in the left and right child nodes.
- H_L and H_R are the entropies of the left and right child nodes.
- N is the total number of samples before the split.

2.3.4 Information Gain

Information gain measures the reduction in uncertainty caused by a split.

It is defined as:

$$IG = H_{\text{before}} - H_{\text{after}}$$

where:

- IG is the information gain.
- H_{before} is the uncertainty before splitting.
- H_{after} is the remaining uncertainty after splitting.

A decision tree selects the split that maximizes information gain, because this split produces the largest reduction in uncertainty and creates more homogeneous child nodes.

2.3.5 Comparison between Gini and Entropy

Here is a summary of how these two methods relate to each other.

Table 1: Comparison of Gini Impurity and Entropy

Aspect	Gini Impurity	Entropy
Definition	Measures the probability of misclassifying a randomly selected sample.	Measures the amount of uncertainty or information in a node.
Formula	$G(S) = 1 - \sum_i p_i^2$	$H(S) = - \sum_i p_i \log_2(p_i)$
Range	0 to 0.5 (binary classification)	0 to 1 (binary classification)
Computation	Simpler to compute because it does not require logarithms.	Slightly more computationally expensive because it uses logarithms.
Sensitivity	Often slightly favors the majority class.	More sensitive to changes in class probabilities.

3 Decision Tree Example: Loan Approval

To illustrate how a decision tree works, consider a financial institution that needs to decide whether to approve or reject a loan application.

The decision is based on three applicant characteristics: Income, Credit score, Current debt.

The target variable is the loan decision: Approve loan, Reject loan.

The training data contains past loan applications with known outcomes, and will look something like this:

Table 2: Sample input data

ID	Income	CreditScore	CurrentDebt	LoanDecision
1	32000	580	18000	Reject
2	45000	640	12000	Reject
3	78000	720	9000	Approve
4	92000	760	15000	Approve
5	51000	690	5000	Reject

The real data set will of course be much larger than the above set, i.e. thousands if not millions of rows and mostly likely more features. The decision tree starts with all applicants in a single node. The tree then evaluates possible questions to determine which split creates the best separation between approved and rejected loans.

Possible split questions include:

- Is credit score greater than a certain threshold?
- Is income greater than a certain threshold?
- Is debt less than a certain threshold?

Each split creates two groups. For example:

Credit Score > 700

creates:

Yes	No
High credit score group	Low credit score group

The tree does not create multiple branches for different features. Instead, each candidate feature is evaluated, and the best binary split is selected.

For each possible split, the tree measures the quality of the separation.

Using Gini impurity:

$$\text{Gini Gain} = G_{\text{before}} - G_{\text{after}}$$

where the split with the largest reduction in impurity is selected.

Using entropy:

$$\text{Information Gain} = H_{\text{before}} - H_{\text{after}}$$

where the split with the largest reduction in uncertainty is selected.

After choosing the best split, the same procedure is repeated within each child node until a stopping condition is reached.

Therefore, both Gini impurity and entropy provide different mathematical criteria for selecting the same decision tree structure:

- Gini impurity selects the split that maximizes impurity reduction.
- Entropy selects the split that maximizes information gain.

3.1 Credit score analysis

For illustration purposes we have a data set of 100 points. The actual data set is provided in a "decision_tree.csv" file. The first 5 lines of that file are actually what was used in table Table 2.

Let us start by calculating the impurity using Gini and the Entropy for our original set.

Entropy	Gini Impurity
0.9974	0.4982

Next will do the same but not for the set with 3 potential splits, and we pick the split that has the highest difference or the min new Impurity. The results are shown below. They are all lower from the original set impurity and the credit score in the lowest in both cases. So that is what we start to split our data on.

Table 3: Freture split comparison table

Feature	BestEntropyThreshold	MinEntropy	BestGiniThreshold	MinGini
Income	66500.000000	0.471892	66500.000000	0.197079
CreditScore	707.500000	0.468180	707.500000	0.179798
CurrentDebt	19500.000000	0.812184	19500.000000	0.399059

Next we actually take a look at the split data. And we can see how the split generated two nodes, one has mostly "Approve" and one has mostly "reject". The ideal goal is to come up with a set that has only one label, a pure node. Now have two branches and we still have two features.

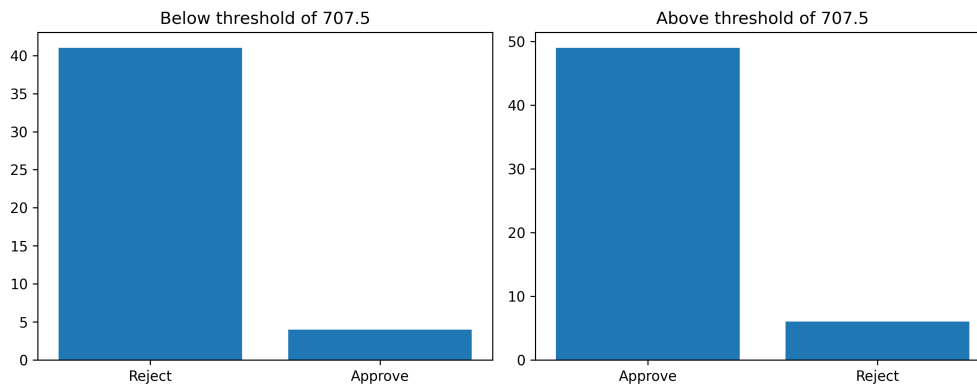


Figure 1: Description of the figure

We want to utilize these features to see if we can get to our goal. Let us first look at the "above threshold" node and let us see what should we use to split it next.

Table 4: Pure Classification Rules

Feature	BestEntropyThreshold	MinEntropy	BestGiniThreshold	MinGini
Income	66500.000000	0.351160	66500.000000	0.147386
CurrentDebt	19500.000000	-0.000000	19500.000000	0.000000

The above table tell us that if we split this node based on current debt we get to two leaf nodes. I.e. the impurity is 0. That is the best outcome we can hope for. Let us see how the two split results look like.

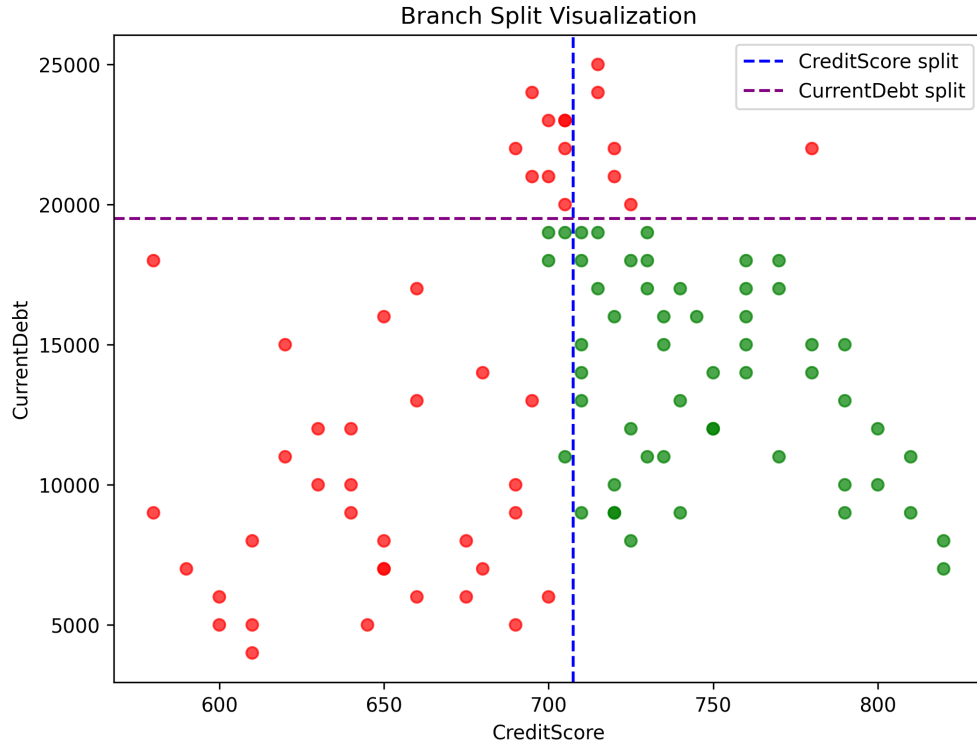


Figure 2: Split based on credit score and current debt

We can clearly see that right side (credit score above threshold) can be split to two clean sets, each has only out label, red or green, i.e approve or deny. This means we figure out the rules on this branch and all we are left with leaf nodes. But we still have to figure out the other branch, credit score < threshold.

Table 5: Pure Classification Rules

Feature	BestEntropyThreshold	MinEntropy	BestGiniThreshold	MinGini
Income	52500.000000	0.278880	68500.000000	0.124242
CurrentDebt	10500.000000	0.340696	17500.000000	0.145620

From the above table, the we get the min entropy if we do the split based on income. So let us do that and see what that split look like.

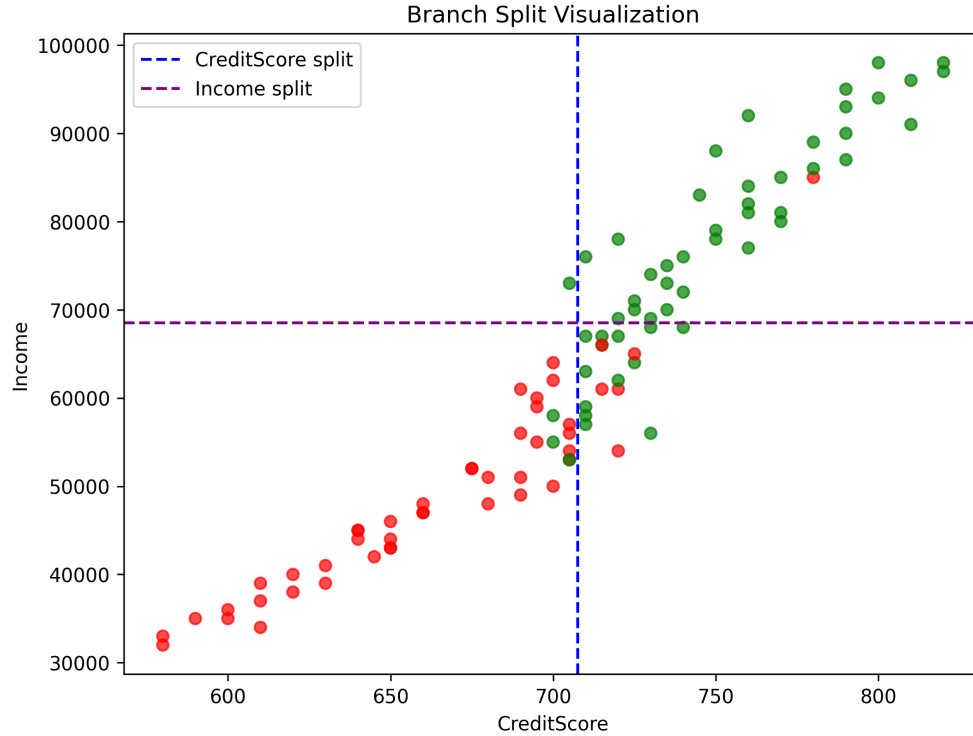


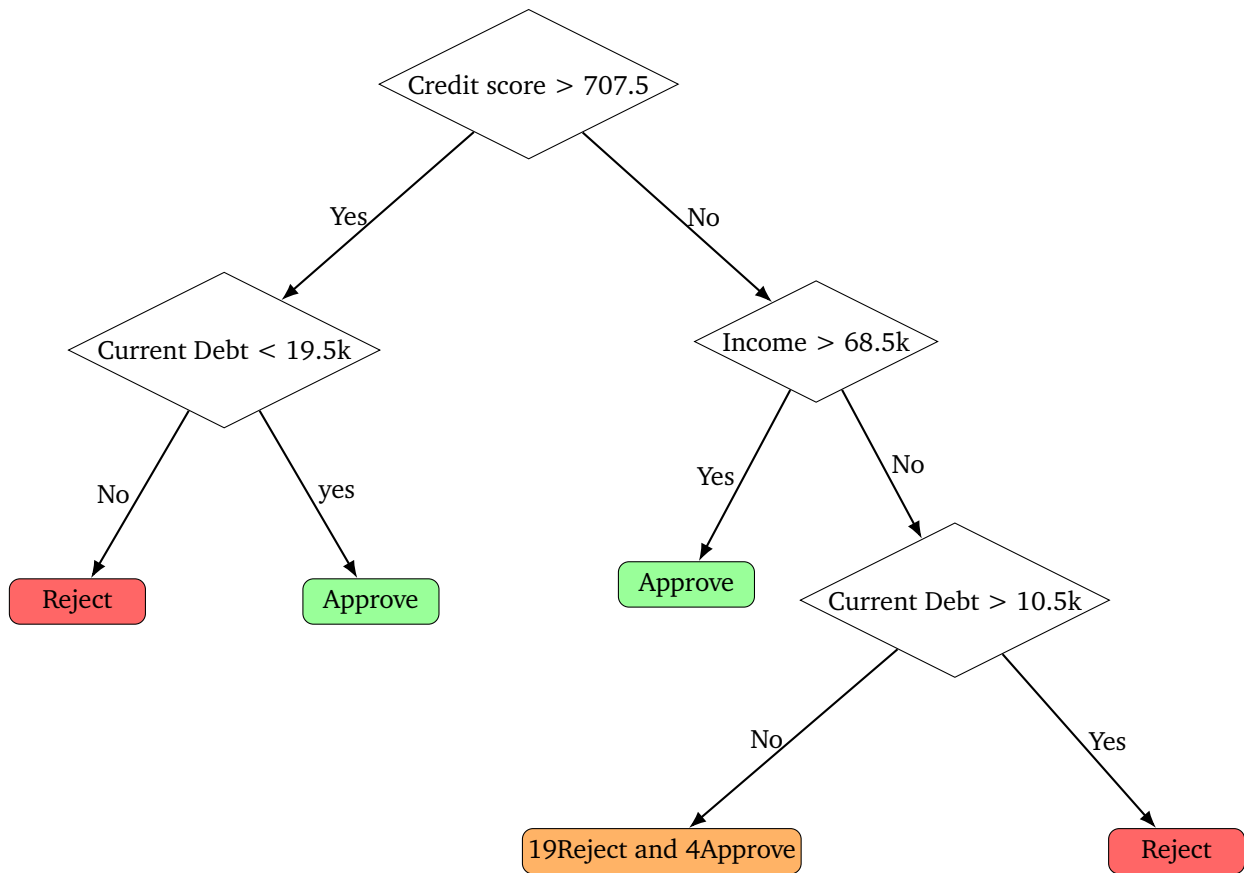
Figure 3: Split based on credit score and income

In the top left quadrant, we have one point only. That means the node is a leaf node and we do not have to do anything about it. The only thing left is the lower left quadrant (below credit score threshold and below income threshold). We still have one feature we have not yet used. So let us see what kind of improvement we can get if we utilize that feature.

Table 6: Pure Classification Rules

Feature	BestEntropyThreshold	MinEntropy	BestGiniThreshold	MinGini
CurrentDebt	10500.000000	0.287318	10500.000000	0.117769

Here I want to note a few things, the entropy and Gini Impurity went down for this last set. They were 0.3591 and 0.1270 respectively. And despite the fact that neither went to zero, we still had a leaf node with class reject. So, now we want to figure out what is the outcome of all this process, we say we have a "routine" that allows us to make consecutive decisions based on the tree below to reject, approve or in one case perform further investigation.



Another important note to make is that the threshold for the same feature will change based on the branch we are on. Note how the current debt had a 19.5 and 10.5k thresholds.

3.2 “It’s better to be lucky than good.”

In many cases the easiest solution is the simplest solution. Inspired by the idea of the decision tree, I decided to look for a simpler threshold on the features. Without any entropy or Gini impurity, I scanned each feature looking for two values lower and upper threshold that might bound all the Approve or Reject target. I.e. can I find, for each feature a point or two where under, above or between I have only reject or approve. It is a long shot but I thought I will try it. And sure enough there was indeed such 3 values associate with these features, shown below. And if I impose these three thresholds I slice my data to two sets, one has only reject (47 out of 100 points) and one has only approve (53 out of 100 points). This approach is not guaranteed and either there or not, but for the easiness of the approach and the reward, it is always worth trying some simpler techniques that have the same idea of the more complicated ones.

Table 7: Pure Classification Rules

Feature	Threshold	Subset	Class	Samples
Income	52500.000000	Below	Reject	30
CreditScore	697.500000	Below	Reject	34
CurrentDebt	19500.000000	Above	Reject	15

The python code I provide show a 3D interactive graph for the above set. It makes more sense to see it in interactive 3D environment than to create a 2D version of the graph.